

数值计算方法

Numerical Computing Method

绪论

华中科技大学 人工智能与自动化学院

- School of AI & Automation.
- Huazhong University of Science and Technology

蔡超

caichao@hust.edu.cn

一、计算方法的基本内容

1、数值逼近

插值法

函数逼近与曲线拟和

数值积分与数值微分

2、数值代数

线性代数问题(方程组和特征值)

非线性方程(组)数值解法

3、常微方程数值解法和偏微方程数值解法

● 高等学校教材

数值分析简明教程

(第二版)

王能超 编著

 高等教育出版社
HIGHER EDUCATION PRESS



普通高等教育“十一五”国家级规划教材

李庆扬

王能超

易大义

编

数值分析

第5版

清华大学出版社

二、计算方法及其意义

许多科学问题的解决都离不开科学计算。本门课程将着重介绍进行科学计算所必须掌握的一些最基本、最常用的算法,并分析其误差。

实际问题→数学模型→问题求解（**数值计算方法**→**程序设计**→**上机计算**求出结果）

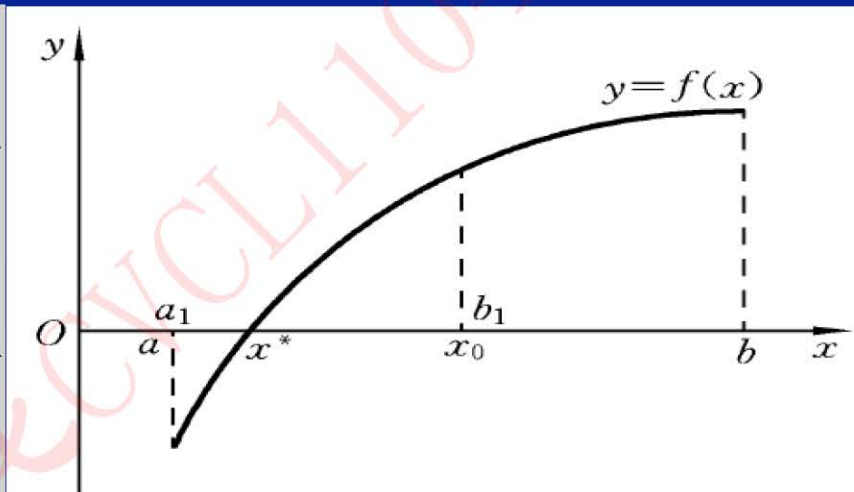
方程求根的二分法

设函数 $f(x)$ 在 $[a, b]$ 上连续, 且

$f(a)f(b) < 0$, 假定 $f(x)$ 在 $[a, b]$ 上有唯一的实根 x^* 。考察有根区间 $[a, b]$, 取中点

$x_0 = (a + b) / 2$ 将它分为两半。然后进行根的搜索

即检查 $f(x_0)$ 和 $f(a)$ 是否同号。



若确定同号, 说明所求的根在 x_0 的右侧, 这时令 $a_1 = x_0, b_1 = b$, 否则 x^* 必在 x_0 左侧, 这时令 $a_1 = a, b_1 = x_0$, 于是我们就得到了一个长度压缩了一半的有根区间 $[a_1, b_1]$, 对于有根区间 $[a_1, b_1]$ 施行同样手续可得到新的有根区间 $[a_2, b_2]$, 反复如此我们可得到一系列的有根区间: $[a, b] \supset [a_1, b_1] \supset [a_2, b_2] \supset \cdots$, 令 $x_k = (a_k + b_k) / 2$, 则二分过程中可获得一个近似根的序列 $x_0, x_1, x_2, \cdots$, 逐步逼近所求的根 x^* 。

方程求根的二分法

算法思想:

对于在区间 $[a, b]$ 上连续不断且 $f(a) \cdot f(b) < 0$ 的函数 $y = f(x)$, 通过不断把函数 $f(x)$ 的零点所在的区间一分为二, 使区间的两个端点逐步逼近零点, 进而得到零点的近似值的方法叫二分法。

不过在实际计算时, 我们不可能也没有必要完成这种无穷过程, 因为计算结果允许带有一定的误差. 由于

$$|x^* - x_k| \leq \frac{1}{2}(b_k - a_k) = \frac{1}{2^{k+1}}(b - a)$$

只要二分足够多次 (即 k 充分大), 便有

$$|x^* - x_k| < \varepsilon$$

这里 ε 为预定精度.

共 104 页

方程求根的二分法

算法 (algorithm) :

(1) 确定区间 $[a, b]$, 验证: $f(a) \cdot f(b) < 0$, 确定精确度 ε

(2) 求区间 (a, b) 的中点 x_1

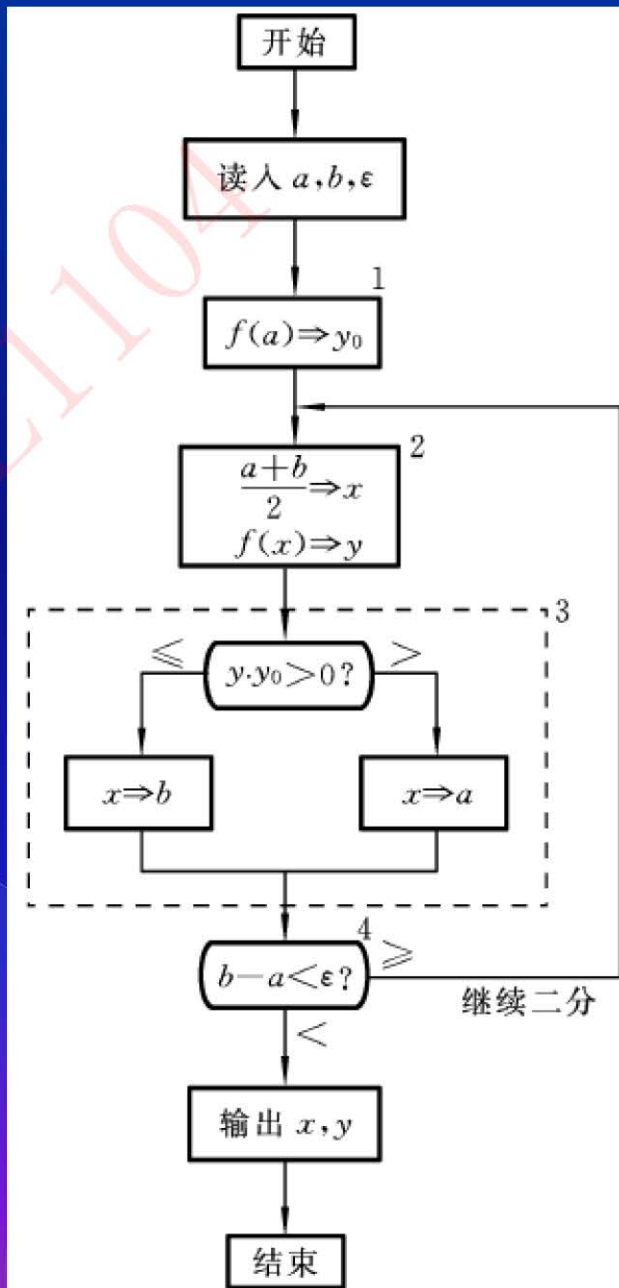
(3) 计算 $f(x_1)$ 若 $f(x_1) = 0$, 则就 x_1 是函数的零点

若 $f(a) \cdot f(x_1) < 0$, 则令 $b = x_1$ (此时零点 $x_0 \in (a, x_1)$)

若 $f(x_1) \cdot f(b) < 0$, 则令 $a = x_1$ (此时零点 $x_0 \in (x_1, b)$)

(4) 判断是否达到精确度 ε

即若 $|a - b| < \varepsilon$, 则得到零点的近似值为 a (或 b), 否则重复 (2) ~ (4)



算法框图:

研究算法的意义

尽管计算机的运算速度高，可以承担大运算量的工作，但这并不意味着我们可随意选择 计算机上的算法，相反的，算法的优劣决定着计算效率的高低，能否正确地制定算法也是科学计算成败的关键。

算法的质量标准

求解一个数学问题，可以采用不同的算法，比如：线性方程组，可用克莱姆法则，高斯消元法等多种方法求解。但是每一种方法的优劣不同，评价一个算法的好坏有以下几个标准：

1) 算法的计算量（时间复杂性）

例1：用克莱姆法则求解一个 n 阶线性方程组时，需要计算 $(n+1)$ 个 n 阶行列式的值。需要做 $n^2 \cdot n!$ 次乘法。设 $n=20$ ，若采用10亿/秒的计算机，要花费三十万年的时间进行计算。若用高斯消元法来求解，采用一个普通的微机，在几秒之内就可得结果。

计算量的大小是衡量一个算法优劣的重要标准。

2) 算法的空间复杂性

当使用计算机求解一个数学问题时，计算程序要占用许多工作单元（内存）。当计算一个大型的数学问题时，内存的消耗量是很大的。因此，算法占用内存数量的多少，是衡量算法优劣的另一个标准。

3) 算法逻辑结构的复杂性

设计算法时应该考虑的另一个因素是逻辑结构问题，虽然计算机能自动执行极其复杂的计算程序，但是计算程序的每个细节都需要编程人员制定，因此算法的逻辑结构应尽量简单，才能使程序的编制、维修和使用比较方便。

4) 算法的精度

以上我们介绍了算法的一些基本概念。下面讨论数值计算中的另一个重要问题——误差。

三、数值计算的误差

误差的来源

- 数值计算中的近似是正常的，计算误差是不可避免的：
 - **模型误差**，为要进行数值计算，我们往往把实际问题归结为数学问题，然后建立起合适的数学模型。这些模型描述的近似必然会产生误差；
 - **观测误差**；
 - **截断误差**(本课考察的主要误差来源)；
 - **舍入误差**(本课考察的主要误差来源)。

截断误差

$$f(x) \approx P_n(x) = f(0) + \frac{f'(0)}{1!}x + \frac{f''(0)}{2!}x^2 + \cdots + \frac{f^{(n)}(0)}{n!}x^n$$

截断误差: $R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}x^{n+1}$

舍入误差

$$R = \pi - 3.14159 = 0.0000026\cdots \text{数字转换、机器数.}$$

误差、有效数字

定义1 绝对误差，简称误差：

设 x^* 为准确值， x 为 x^* 的一个近似值，称 $e^* = x - x^*$ 为近似值 x 的绝对误差，简称误差。

误差是有量纲的量，量纲同 x^* ，它可正可负。误差一般无法准确计算，只能根据测量或计算情况估计出它的绝对值的一个上限，这个上界称为近似值 x 的**绝对误差限**，简称**误差限**，或称为**精度**，记为 ε 。

对于一般情形 $|x - x^*| \leq \varepsilon$ ，即

$$x - \varepsilon \leq x^* \leq x + \varepsilon$$

工程中常表示为

$$x^* = x \pm \varepsilon$$

例如，有两个量 $x = 10 \pm 1, y = 1000 \pm 1$ ，则

$$x = 10, \varepsilon_x = 1; y = 1000, \varepsilon_y = 1。$$

相对误差:

定义2 近似值的误差 e 与准确值 x^* 的比值

$$e_r = \frac{e^*}{x^*} = \frac{x - x^*}{x^*}$$

称为近似值 x 的相对误差, 记作 e_r 。

在计算中, 由于真值 x^* 总是不知道的, 通常取

$$e_r = \frac{e}{x} = \frac{x - x^*}{x}。$$

相对误差限: 相对误差绝对值的上界, 记作 ε_r , 即

$$\varepsilon_r = \frac{\varepsilon}{|x|} \geq \frac{|x - x^*|}{|x|} = |e_r|$$

上例中 $\frac{\varepsilon}{|x|} = 10\%$ 与 $\frac{\varepsilon}{|y|} = 0.1\%$ 分别为 x 与 y 的对误差

限, 可见 y 近似 y^* 的程度比 x 近似 x^* 的程度好。

定义3: 若近似值 x 的误差限是某一位数字的半个单位,该位到 x 的第一位非零数字共有 n 位,就说 x 有 n 位有效数字.

即
$$x = \pm 0.a_1a_2 \cdots a_n \cdots a_k \times 10^m$$

其中 $a_1 \neq 0$. 并且

$$|x - x^*| \leq \frac{1}{2} \times 10^{m-n}$$

例1 下列数按四舍五入原则得到的5位有效数字的近似数分别是?

187.9325

0.03785551

8.0000033

2.7182818

解 :

187.93

0.037856

8.0000

2.7183

例2 考察三位有效数字重力加速度 g ,

若以 m/s^2 为单位, $g \approx 9.80 \text{m/s}^2$,

$$|g - 9.80| \leq \frac{1}{2} \times 10^{-2},$$

$$m = 1, n = 3 \text{ 绝对误差限 } \varepsilon = \frac{1}{2} \times 10^{-2}.$$

若以 km/s^2 为单位, $g \approx 0.00980 \text{m/s}^2$,

$$|g - 0.00980| \leq \frac{1}{2} \times 10^{-5},$$

$$m = -2, n = 3. \text{ 绝对误差限 } \varepsilon = \frac{1}{2} \times 10^{-5}.$$

而相对误差限相同:

$$\varepsilon_r^* = 0.005/9.80 = 0.000005 / 0.00980.$$

$$\varepsilon = \frac{1}{2} \times 10^{m-n}$$

应用中看
小数点后
有几位就
行了

数值运算的误差估计

四则运算，设 x_1, x_2 为近似值, x_1^*, x_2^* 为准确值，则误差限：

$$\varepsilon(x_1 \pm x_2) = \varepsilon(x_1) + \varepsilon(x_2),$$

$$\varepsilon(x_1 x_2) \approx |x_1| \varepsilon(x_2) + |x_2| \varepsilon(x_1),$$

$$\varepsilon(x_1 / x_2) \approx \frac{|x_1| \varepsilon(x_2) + |x_2| \varepsilon(x_1)}{|x_2|^2}.$$

$$\begin{aligned} \varepsilon(f(x_1, x_2 \dots x_n)) &= f(x_1, x_2 \dots x_n) - f(x_1^*, x_2^* \dots x_n^*) \\ &\approx \left[\sum_{i=1}^n \left| \left(\frac{\partial f}{\partial x_i} \right)^* \right| \varepsilon(x_i^*) \right] \end{aligned}$$

$$\left| \frac{f(x) - f(x^*)}{f(x^*)} \right| \approx \left| \frac{f'(x^*) \varepsilon(x^*)}{f(x)} \right|$$

误差定性分析、避免误差危害

一、病态问题与条件数

考虑计算函数值问题,

$$\left| \frac{f(x) - f(x^*)}{f(x)} \right| \bigg/ \left| \frac{\Delta x}{x} \right| \approx \left| \frac{xf'(x)}{f(x)} \right| = C_p,$$

C_p 称为计算函数值问题的条件数.

例如 $f(x) = x^{10}$, $C_p = 10$, $f(1) = 1$, $f(1.02) \approx 1.24$, 自变量相对误差为2%, 函数值相对误差为24%.

一般 $C_p \geq 10$ 认为是病态.

其他计算问题也要考虑条件数, 考虑是否病态.

二、算法的数值稳定性

考虑初始数据误差在计算中的传播问题.

定义4 一个算法若输入数据有 误差,而在计算过程中舍入误差不增长,则称此算法是数值稳定 的,否则是不稳定的.

避免误差危害的若干原则

除了分清问题是否病态和算法是否数值稳定外, 还要考虑避免误差危害和防止有效数字损失的如下原则.

一、避免误差危害的若干原则

1、要避免除数绝对值远远小于被除数绝对值的除法。

用绝对值小的数作除数舍入误差会增大, 如计算 x/y , 若 $0 < |y| \ll |x|$, 则可能对计算结果带来严重影响, 应尽量避免。

2、要避免两相近数相减

在数值中两相近数相减有效数字会严重损失。例如， $x=532.65$ ， $y=532.52$ 都具有五位有效数字，但 $x-y=0.13$ 只有两位有效数字。通过改变算法可以避免两相近数相减。

例： 计算 $A=10^7(1-\cos 2^\circ)$ 。由于 $\cos 2^\circ=0.999\ 4$ ，直接计算，得

$$A=10^7(1-\cos 2^\circ)=10^7(1-0.999\ 4)=6\times 10^3,$$

只有一位有效数字；若利用 $1-\cos x=2\sin^2\frac{x}{2}$ 计算，则有

$$A=10^7(1-\cos 2^\circ)=2\times (\sin 1^\circ)^2\times 10^7=6.13\times 10^3,$$

具有三位有效数字(这里 $\sin 1^\circ=0.017\ 5$)。

3. 防止‘大数’吃‘小数’

例2 仿计算机在3位十进制下,

$$123 + \overbrace{0.01 + 0.01 + \cdots + 0.01}^{100\text{项}} \neq \overbrace{0.01 + 0.01 + \cdots + 0.01}^{100\text{项}} + 123$$

4、注意简化计算步骤，减少运算次数

减少算术运算的次数不但可减少计算机的计算时间，还能减少误差的积累效应。使参加运算的数字精度应尽量保持一致，否则那些较高精度的量的精度没有太大意义。

作业 P11, 2, 8, 13.